ECE 284 Project: Reconfigurable AI Accelerator based on 2D Systolic Array

Group Hachimi
Ziyi Xiao A69035869
Hamad Alajeel A16369878

***Abstract*—In this project, improvements to various designs of the reconfigurable AI accelerators based on the 8×8 2D systolic array were proposed to improve the efficiency and versatility of the designs to be used in the computation of output feature maps of Convolutional layers in CNN's. 2 and 4-bit quantization aware VGGNet16's were trained to evaluate the quantization errors introduced by our implementation of these AI accelerators, as well as the new features we have equipped them with. The improvements we propose for the base designs are as follows: allowing the designs to be able to have reconfigurable activation functions in the SFP, mapping various types of CNN network to our hardware, implementing pipelining when accelerator reading and writing the data, and finally implementing in-place accumulation for the output stationary design to reduce latency and lower energy consumption.**

I. Implementation

A. VGG Model Training

Layer feature[27] was chosen to shrink the input and output channel numbers to 8 and avoid early feature map loss. Quantization error was obtained first by multiplying activation and weight of layer feature[27], and then passed the results to ReLU, which were further compared with the pre-hooked value of next layer. The results of modified VGG model was shown in Table I.

TABLE I
Training Results of VGG16

	Accuracy	Quantization Error
VGG16	90.08%	1.05×10^{-7}
VGG16 (2bit)	85%	1.42×10^{-6}

B. Hardware Design with Verilog

1) Vanilla Version: The systolic array should consist of the following components. An 8×8 array with multiplication and accumulation (MAC) units to perform convolution. An 32-bit input SRAM that loaded with activation and weight data. An L0 FIFO that passed the activation and weight to the array for further computation. An Output FIFO that transferred the partial sum to the 128-bit output SRAM. Moreover, there was special function processor (SFP) that perform ReLU operation. The overall structure of Vanilla Version was shown in Figure 1 .

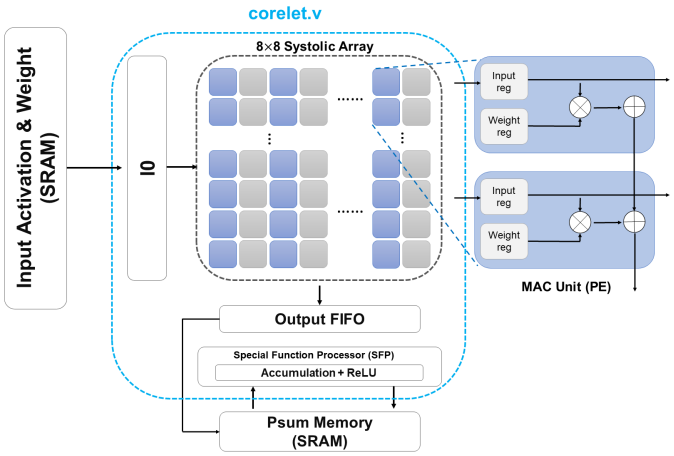


Fig. 1. Overall Structure of Vanilla Version 2D Systolic Array

2) Multi-bit Lane Reconfigurable Version: For this version of the Systolic array, our design enables each PE to compute the MAC operation for two input channels instead of 1 while keeping the array size the same, 8 rows and 8 columns. This is achieved when the input activations are set to 2 bits, and the weight activations are kept as 4 bits. Pairs of weights representing subsequent input channels are loaded to each of the PE's in the same fashion as the vanilla version of the systolic array and loaded into separate 4 bit registers.

In the case where our activations are 2 bits for each channel, the 2 lsb bits represent the activation for input channel, i, and the 2 msb bits represent input channel $i + 1$. By multiplying them with their respective weights stored in each weight register and accumulating them, each PE can now process two input channels at once, allowing the array size to remain the same while increasing throughput.

This design is also able to be reconfigured to allow for 4bit activations to be processed. In this case, the pairs of 4bit weights are identical, and represent the same input channel. Multiplying the 2 msb bits and 2 lsb bits of the 4bit activation with each of the weights, bit shifting by 2 for the msb product, merging, and finally accumulating with the psum outputs the equivalent result as the vanilla version.

3) Weight/Output Stationary Reconfigurable Version: In an output-stationary 2D systolic array, each MAC row corresponds to a specific spatial location (feature) in the output feature map, while each column corresponds to one of the

output channels being computed for that location. The time for loading weights and activations to the l0 fifo and northern fifo is represented by the input channel of which we are beginning computations for. Each MAC row holds the sliding convolution window over the activation map that produces its respective output feature, so the first row computes $O_{nij} = 0$, the eighth row computes $O_{nij} = 7$, and the time step inside each processing element indicates which activation value is being multiplied by its corresponding k_{ij} weight at that moment. The weight file is arranged so that each row contains all nine kernel elements k_{ij} (from k_0 to k_8) for a given output channel, with channels indexed from 0 to 8 in this test. The output text file follows the same structure: each row represents a different spatial output O_{nij}, and the testbench verifies correctness from output feature 7 down to 0, evaluating only the first eight spatial outputs for the first eight output channels out of the full 64-channel design.

C. Testbench Design and Verification

To verify the functionality of our 2D systolic array, a testbench was required to act as a top level controller that generated all the control signals and organized data flows. The testbench first read the activation and weight text files and passed those values to memory. The activation and weight data were then written to L0 FIFO and loaded to the MAC array for execution. After all execution was done, OFIFO would read the results and stored to output SRAM. The SFP would then accumulate the partial sums based on the address file and perform ReLu to get the final output feature maps. Finally, computation results from our systolic array would be compared with expected output to evaluate the correctness of our design. The work flow of the test bench was shown in Fig 2.

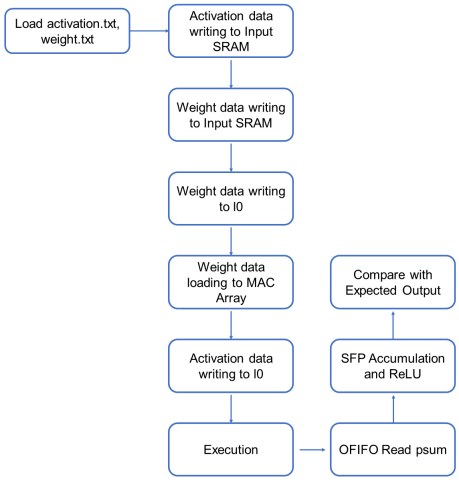


Fig. 2. Work Flow of Testbench

To fully validate our design, three sets of activation and weight data were tested. Our testbench successfully complete all the procedures in Fig 2 and output the correct sum with no errors.

II. Mapping on FPGA and Simulation Results

To evaluate the performance and efficiency of our designed 2D systolic array, Quartus Prime was used to simulate actual mapping on FPGA. We choose to map corelet.v onto the FPGA via Quartus Prime. The detailed structure of corelet.v was shown in Fig 3. To ensure a consistent evaluation standard, all tests are conducted on the Cyclone IV GX EP4CGX150DF31I7AD platform. Frequency was tested at the slow corner and the power consumption was measured with a 20% input activity factor.

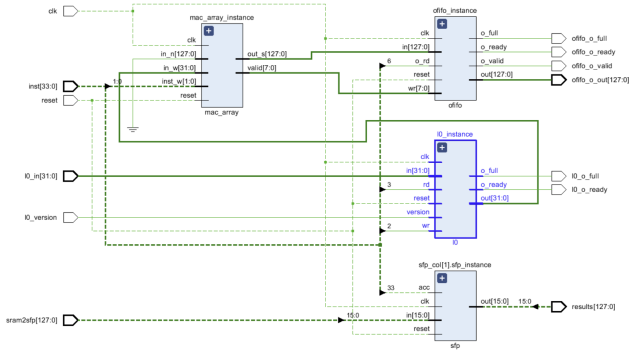


Fig. 3. Structure of corelet.v

The simulation results of Quartus Prime were shown in Table II.

TABLE II
Simulation Results from Quartus Prime

	Vanilla	Bit Reconfigurable	Weight/Ouput Reconfigurable
Total Registers	12098	14658	15161
Total Logical Elements	17233	22316	23000
Frequency	127.5MHz	92.35MHz	88.97MHz
Dynamic Thermal Power	33.48mW	22.16mW	27.36mW
GOPs	16.32	11.82	11.39
GOPs/W	487.45	533.39	426.45

III. Alphas

To fully exploit the structure of designed 2D systolic array, four different Alphas were implemented to optimize the operations of our designs and add aditional functionalities.

A. Alpha 1: Alternative Activation Functions

In order for our designs to be more versatile and applicable to a wider variety of Convolutional neural networks, we allowed our designs to have reconfigurable activation functions in the SFP modules. We applied two new and important activation functions used by CNN's in different applications. They are the Leaky ReLU activation function and the Soft shrink. We chose these functions to implement in the SFP for these reasons: Leaky ReLU is an important activation function used by some CNN architectures to remedy the phenomenon of dying neurons. This is accomplished by allowing a small,

non-zero gradient for negative output features which enhances the stability of training and feature learning. We chose to apply soft shrink because it is an important activation function used for CNN architectures in the medical field. For example, a CNN called ID3RSNet, used for EGG analysis uses soft shrink activation functions to eliminate noise and adaptively eliminate redundant information in the activation maps [1]. Thus, giving the designs these two new activation functions allows them to be utilized by a wider variety of networks, not just ones using ReLU.

1) Implementation in SFP: To allow the SFP to be reconfigurable, we equipped it with a 2 bit wide control input to be used in selecting which activation function to use.

$$\text{output}(\text{act}, x) = \begin{cases} x, & \text{if act} = 0, \\ \text{ReLU}(x), & \text{if act} = 1, \\ \text{LeakyReLU}(x), & \text{if act} = 2, \\ \text{SoftShrink}(x), & \text{if act} = 3\ . \end{cases} \tag{1}$$

2) Implementation of Leaky ReLU: The Leaky ReLU activation function can be implemented before dequantizing the quantized output feature map by approximating the negative region of Leaky ReLU using bit shifting. If the output feature is negative, bit shifting by 3 is equivalent to division by 8 which is equal to multiplication by 0.125. 0.125 is approximately 0.1 which means that applying this would only introduce negligible error to the resulting output features after activation.

3) Implementation of SoftShrink: The SoftShrink activation function is defined as the following:

$$\text{SoftShrink}(x; \lambda) = \begin{cases} x - \lambda, & \text{if } x > \lambda, \\ 0, & \text{if } -\lambda \leq x \leq \lambda, \\ x + \lambda, & \text{if } x < -\lambda\ . \end{cases} \tag{2}$$

zeroing out the symmetrical region around the point 0 aids CNN's in eliminating noise from input data, allowing better feature extraction. λ is a floating point number chosen between (0,1]. To apply this activation function in the quantized output feature domain, we are required to find an integer, c, which thresholds the output features in such a way such that we achieve this condition:

$$\begin{aligned} T &< \left| (c+1)\, \frac{\alpha_{\text{act}} \alpha_w}{(2^{\alpha_a} - 1)(2^{\alpha_w - 1} - 1)} \right| \\ \text{and} \quad T &\geq \left| c\, \frac{\alpha_{\text{act}} \alpha_w}{(2^{\alpha_a} - 1)(2^{\alpha_w - 1} - 1)} \right| \end{aligned} \tag{1}$$

We call this integer value λ_{int}, and we compute it using this equation:

$$\lambda_{\text{int}} = \text{round}\left(\frac{\lambda}{\Delta_{\text{out}}} \right) \tag{3}$$

where,

$$\Delta_{\text{out}} = \Delta_{\text{act}} \cdot \Delta_{\text{weight}} = \frac{\alpha_{\text{act}} \alpha_w}{(2^{\alpha_a} - 1)(2^{\alpha_w - 1} - 1)} \tag{4}$$

We then applied the required thresholding and operations to our output feature maps according to the definition of the function in the SFP using this derived value for λ_{int}.

4) Experiments for The Two New Activation Functions: In order to experiment with how effective our application of the activation functions in the quantized output feature domain are, we trained two quantized VGGNet16 models which were exactly the same as the original quantized VGGNet16 model, except for one layer. For the first convolutional layer, we removed Batchnorm and ReLU and replaced them with each of the 2 activation functions for each of the models, so VGG16 (Leaky ReLU)'s first Convolutional layer was succeeded by a Leaky ReLU activation and VGG16 (SoftShrink)'s first Convolutional layer was succeeded with a SoftShrink activation. After training each of the models for 600 epochs with the same hyperparameters, we evaluated the quantization error introduced by applying our methods for implementing the two new activation functions in the quantized output domain.

TABLE III
QUANTIZATION ERRORS OF APPLYING LEAKY RELU & SOFTSHRINK

Model	VGG16 (Leaky ReLU)	VGG16 (SoftShrink)
Accuracy	86%	87%
Quantization Error	0.1338	0.0968
no.# Input Channels	3	3
no.# Output Channels	64	64
Extracted Layer	First Convolutional Layer	First Convolutional Layer

B. Alpha 2: Deployment of ResNet20

A general AI accelerator should support multiple types of neural networks rather than VGG16 and has much more channels. Hence in Alpha 2, a 4-bit quantization-awared-training ResNet20 model with 80% sparsity was trained. The comparison of VGG16 model used in Vanilla version and ResNet20 in Alpha 2 were shown in Table IV.

TABLE IV
COMPARISON BETWEEN VGG16 AND RESNET20

	VGG16	ResNet20
Accuracy	90.08%	87.59%
Quantization Error	1.05×10^{-7}	4.31×10^{-8}
no.# Output Channels	8	16
Extracted Layer	feature[27]	layer1[2].conv1

To implement a 16×16 layer into our 8×8 array, the input layers were split it into 2×2 tiles.

C. Alpha 3: Processing Cycle Reduction

Pipelining was an important technic in modern architecture which greatly improved the operation efficiency. In this Alpha, we adapted the idea of pipelining to our hardware, allowing the systolic array to combine weight memory write to l0 and weight loading to kernel at the same time. This reduces the total duration of completing the whole operation of inference

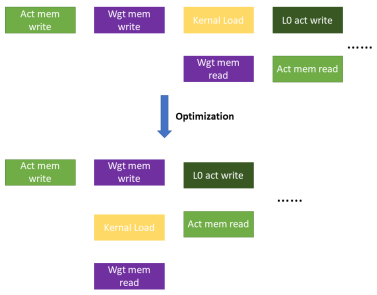


Fig. 4. Optimization Procedures of Alpha 3

using our accelerator. The optimization procedures of this alpha were shown in Fig 4.

D. Alpha 4: In-place Accumulation, Output Stationary Design

In the Output Stationary design of the 2D Systolic Array, the time used for loading weights and activations to the northern input fifo and l0 input fifo represents the input channel currently being processed. During each iteration of loading weights and activations and computing calculations in the PE's, the PSUMs represent the current input feature being processed for the oc'th output channel and the o_{nij}'th feature. To calculate the final output feature, these PSUMs need to be accumulated in the SFP. However, computing these PSUMs for each input channel, transporting them to the PSUM SRAM, and further transporting them to the SFP uses many clock cycles and requires sending data across the bus connecting the ofifo to the PSUM SRAM much more. To optimize the amount of clock cycles required by this design to transport data from the mac array to the PSUM SRAM and from the PSUM SRAM to the SFP modules, we have applied in-place accumulation in the PE's such that after all the input channel computations have been processed, the PE's would store the final output features. This would reduce the total amount of times required to send data to the PSUM SRAM and to send data from the PSUM SRAM to the SFP's from the number of input channels being processed to 1. Furthermore, by using the bus connecting the PSUM SRAM to the ofifo and SFP units much less, we reduce the amount of energy consumed by transporting data, as well as possibly increasing the maximum frequency our design is able to attain. We calculate our total reduction in clock cycles as follows:

No In-place Accumulation:

Sending PSUM's to Ofifo then PSUM SRAM

$$= \text{(input channels) x (number of output features) + 1} \quad (5)$$

Sending data to sfp for accumulation and activation

$$= \text{(input channels) + 1} \quad (6)$$

Using In-place Accumulation:

Sending PSUM's to Ofifo then PSUM SRAM

$$= \text{(number of output features) + 1} \quad (7)$$

Sending data to sfp for accumulation and act.

$$= 1 \quad (8)$$

Total Reduction of Clock Cycles:

$$\Big[(\text{input channels}) \times (\#\text{ output features}) + (\text{input channels}) + 2\Big] - \Big[(\#\text{ output features}) + 2\Big]$$
$$= (\text{input channels} - 1) \times (\#\text{ output features}) + (\text{input channels}) \quad (9)$$

To implement in-place accumulation, a control bit was introduced named *hold_cq* which when toggled to 1 does the following: if the control bit named *mac_tile_version* used for reconfiguring the output stationary design between the vanilla version and output stationary is set to 1 (output stationary version), and reset is toggled to 1 to flush out the contents in the ififo's and set *b_q* & *a_q* registers in the PE's to 0, the contents stored in *c_q* are kept the same. This allows for accumulation across input channels.

IV. Conclusion

We have pushed our designs to become more versatile and optimal by adding more features, as well as introducing necessary optimizations that lead to less clock cycles to complete operations as well as reduce the designs' overall energy consumption. We allowed the activation functions that could be implemented by our designs to become more versatile and reconfigurable using a control bit named act. Two new activation functions were implemented: Leaky ReLU and SoftShrink. These are used in various CNN architectures, and allow our design to be utilized by a wider variety of CNN's. We have also implement multiple CNN network to our AI accelerator to evaluate the generosity of designed hardware. Moreover, we adapted the idea of pipelining to the systolic array that allowing reading and writing into l0 at the sametime, which greatly reduced processing time. Finally, we have implemented in-place accumulation for the output stationary design which reduces the number of clock cycles required to complete the operation by (input channels - 1) x (number of features) + (input channels). These optimizations lower latency and energy consumption while increasing the designs' versatility.

References

[1] X. Feng, Z. Guo, and S. Kwong, "Id3rsnet: cross-subject driver drowsiness detection from raw single-channel eeg with an interpretable residual shrinkage network," *Frontiers in Neuroscience*, vol. Volume 18 - 2024, 2025. [Online]. Available: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2024.1508747